

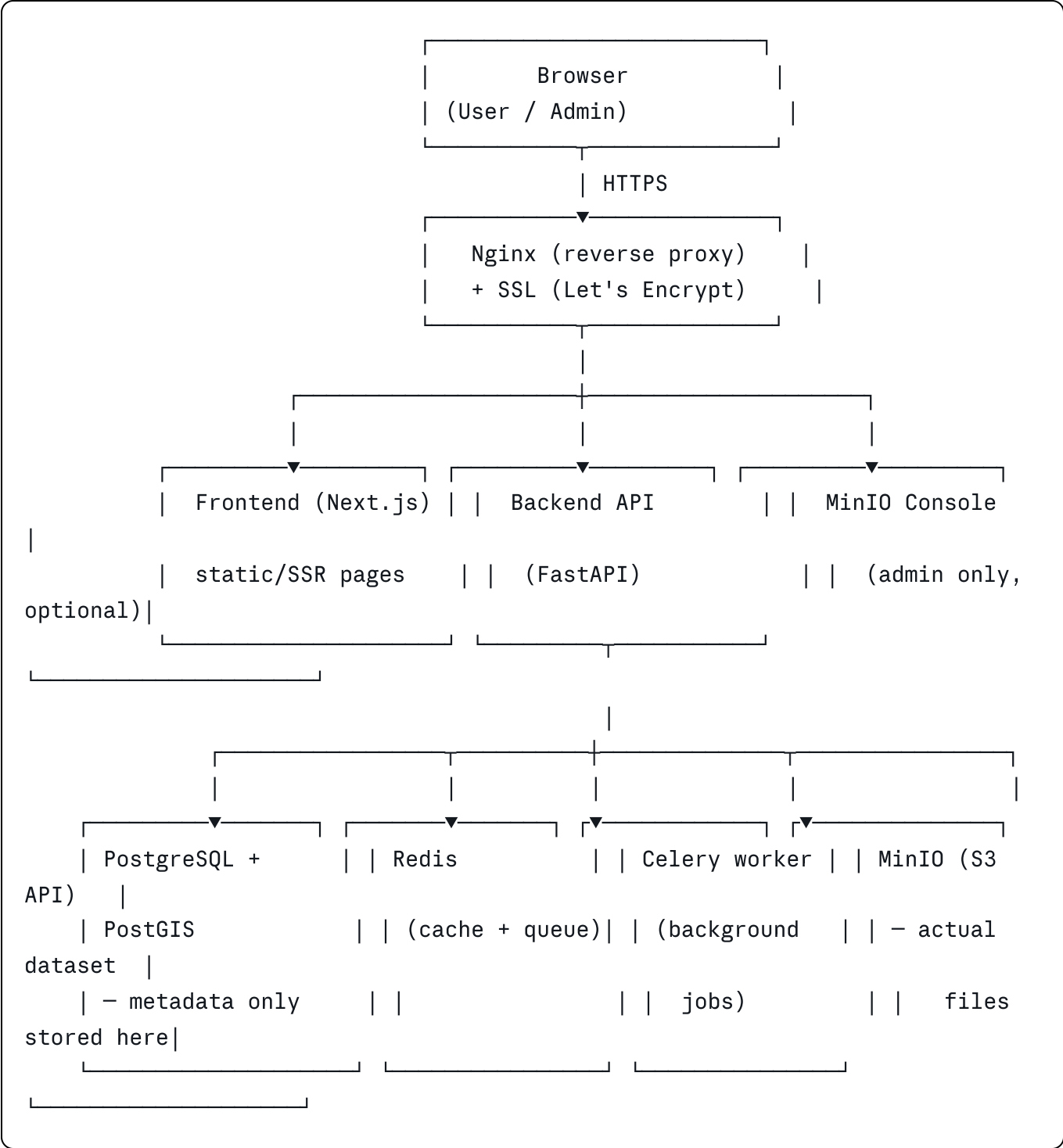
# Environmental & Oceanographic Data Platform — System Architecture

## ১. উদ্দেশ্য ও স্কোপ

এই প্ল্যাটফর্মে থাকবে:

- Public pages: Home, About, Contact
- Data Catalog (search + query + download request)
- Visualization module (temporal + spatial: time-series, interactive maps, interpolated surfaces)
- User Dashboard
- Admin Dashboard (dataset management + request approval)
- Download request → admin approval workflow
- Storage: Hostinger VPS (MinIO দিয়ে শুরু), পরে cloud object storage এ scale করার সক্ষমতা

## ২. High-Level Architecture Diagram



সবগুলো Docker container হিসেবে একই VPS-এ চলবে, `docker-compose` দিয়ে ম্যানেজড।

### ৩. Component-ভিত্তিক বিস্তারিত

#### ৩.১ Frontend (Next.js / React)

পেজগুলো:

- `/` → Home
- `/about`
- `/contact`
- `/catalog` → Data Catalog (search + filter + request download)
- `/visualize` → Visualization module
- `/dashboard` → User Dashboard (নিজের request status, approved download link)
- `/admin` → Admin Dashboard (dataset management + request approval)

Frontend শুধু API কল করে backend থেকে data নেয় — কোনো business logic frontend এ থাকবে না, যাতে ভবিষ্যতে mobile app বা অন্য client যোগ করাও সহজ হয়।

#### ৩.২ Backend API (FastAPI)

Layered structure:

```
backend/  
├─ routers/  
│   ├── auth.py  
│   ├── catalog.py           (search/query dataset metadata)  
│   ├── datasets.py         (admin: add/update/delete dataset)  
│   ├── requests.py         (download request + approval)  
│   ├── visualize.py        (query → data → chart/map data)  
│   └─ files.py             (presigned URL generation)  
├─ services/  
│   ├── storage_service.py   (MinIO/S3 wrapper - boto3)  
│   ├── query_service.py     (DuckDB/Postgres query logic)  
│   └─ notification_service.py (email on request/approval)  
├─ models/                   (SQLAlchemy ORM models)  
├─ schemas/                   (Pydantic request/response schemas)  
└─ worker/                   (Celery tasks - heavy jobs)
```

#### ৩.৩ Database — PostgreSQL + PostGIS

**কেন PostGIS লাগবে:** আপনার dataset গুলোর spatial extent (bounding box / point location) থাকবে, PostGIS দিয়ে "এই এলাকার মধ্যে কোন dataset আছে" — এই ধরনের spatial query fast করা যায়।

#### মূল টেবিল স্কিমা:

```
sql
```

-- ইউজার ও রোল

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  full_name TEXT,  
  role TEXT NOT NULL DEFAULT 'user',    -- 'user' | 'admin'  
  organization TEXT,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

-- dataset এর মূল metadata

```
CREATE TABLE datasets (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name TEXT NOT NULL,  
  description TEXT,  
  category TEXT,                -- 'oceanographic' | 'meteorological' | 'geo'  
  variables JSONB,              -- [{name, unit, description}, ...]  
  spatial_extent GEOMETRY(Polygon, 4326), -- PostGIS bounding box  
  temporal_start DATE,  
  temporal_end DATE,  
  license TEXT,  
  owner_id UUID REFERENCES users(id),  
  status TEXT DEFAULT 'active',   -- 'active' | 'archived'  
  created_at TIMESTAMPTZ DEFAULT now(),  
  updated_at TIMESTAMPTZ DEFAULT now()  
);  
CREATE INDEX idx_datasets_spatial ON datasets USING GIST (spatial_extent);
```

-- dataset এর ভেতরের individual files (একটা dataset এ একাধিক file থাকতে পারে)

```
CREATE TABLE dataset_files (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  dataset_id UUID REFERENCES datasets(id) ON DELETE CASCADE,  
  file_name TEXT NOT NULL,  
  storage_backend TEXT NOT NULL,    -- 'minio_vps' | 'aws_s3' | 'gcs'  
  storage_bucket TEXT NOT NULL,  
  storage_key TEXT NOT NULL,        -- object key/path  
  file_format TEXT,                 -- 'csv' | 'netcdf' | 'parquet' | 'geotiff'  
  file_size_bytes BIGINT,  
  checksum TEXT,  
  spatial_extent GEOMETRY(Polygon, 4326),  
  temporal_start DATE,  
  temporal_end DATE,  
  version INT DEFAULT 1,  
  uploaded_by UUID REFERENCES users(id),  
  uploaded_at TIMESTAMPTZ DEFAULT now()  
);
```

-- download request/approval workflow

```
CREATE TABLE download_requests (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id),  
  dataset_id UUID REFERENCES datasets(id),  
  file_ids UUID[],                -- কোন কোন file চাওয়া হয়েছে  
  purpose TEXT,                   -- কেন লাগবে, ইউজার লিখবে
```

```

status TEXT DEFAULT 'pending',    -- 'pending' | 'approved' | 'rejected'
requested_at TIMESTAMPTZ DEFAULT now(),
reviewed_by UUID REFERENCES users(id),
reviewed_at TIMESTAMPTZ,
review_comment TEXT,
expires_at TIMESTAMPTZ            -- approved হলে কতদিন পর্যন্ত download link v
);

-- download activity log (audit trail)
CREATE TABLE download_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  request_id UUID REFERENCES download_requests(id),
  user_id UUID REFERENCES users(id),
  file_id UUID REFERENCES dataset_files(id),
  downloaded_at TIMESTAMPTZ DEFAULT now(),
  ip_address TEXT
);

-- visualization/query activity log (analytics ও optimization এর জন্য)
CREATE TABLE query_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  dataset_id UUID REFERENCES datasets(id),
  query_params JSONB,
  action TEXT,                    -- 'search' | 'visualize' | 'download_requ
  created_at TIMESTAMPTZ DEFAULT now()
);

```

### ৩.৪ Storage — MinIO (S3 API)

Bucket organization:

```

s3://ocean-data/
  raw/{dataset_id}/{file_id}_{original_name}
  processed/{dataset_id}/{file_id}.parquet    ← query-optimized কপি
  (visualization এর জন্য দ্রুত)
  previews/{dataset_id}/thumbnail.png         ← catalog page এ ছোট preview

```

`raw/` তে original ফাইল (NetCDF/CSV যা-ই upload হোক), `processed/` এ backend automatically একটি optimized version বানিয়ে রাখবে (visualization/query fast করার জন্য) — এটা Celery background job দিয়ে হবে upload এর পরপরই।

### ৩.৫ Redis + Celery — Background Job Processing

কোন কাজগুলো async/background এ হবে (user কে সাথে সাথে wait করাবে না):

- Dataset upload হওয়ার পর raw file থেকে processed/optimized ফাইল তৈরি (CSV→Parquet, ইত্যাদি)
- Spatial interpolation (visualization এর জন্য ভারী compute)
- Large download request এ zip বানানো (একাধিক ফাইল একসাথে চাইলে)
- Email notification পাঠানো (request submit/approve হলে)

Flow:

## 8. মূল Workflow গুলো ধাপে ধাপে

### 8.১ Data Catalog Search & Query

1. User /catalog পেজে যায়
  2. Filter দেয়: category, date range, spatial area (map এ box আঁকে), keyword
  3. Frontend → API call: GET /api/datasets/search?category=oceanographic&start=2024-01-01&bbox=...  
category=oceanographic&start=2024-01-01&bbox=...
  4. Backend: PostGIS দিয়ে spatial\_extent n bbox এবং temporal\_start/end ফিল্টার করে datasets টেবিল থেকে match বের করে
  5. Result: matching dataset list (metadata only, ফাইল না) ফেরত আসে
  6. User একটা dataset select করে বিস্তারিত দেখে, এর ভেতরের file list দেখে (dataset\_files টেবিল থেকে)

এখানে লক্ষ্যণীয়: কোনো ধাপেই object storage (MinIO) touch হয়নি — পুরো search শুধু PostgreSQL metadata দিয়ে হয়েছে, তাই এটা দ্রুত।

### 8.২ Download Request → Approval Workflow

1. User নির্দিষ্ট dataset/file বেছে "Request Download" বাটনে ক্লিক করে, একটা purpose লিখে
  2. POST /api/requests → download\_requests টেবিলে নতুন row (status='pending')
  3. Celery task: admin কে email notification পাঠায়
  4. Admin dashboard এ "Pending Requests" লিস্টে দেখা যায়
  5. Admin Approve/Reject করে (+ optional comment)
  6. Approve হলে:
    - a. status = 'approved', expires\_at সেট হয় (যেমন এখন থেকে ৭ দিন)
    - b. Celery task: user কে email notification পাঠায়
  7. User dashboard এ গিয়ে "Download" বাটনে ক্লিক করে
  8. Backend সেই মুহূর্তে MinIO/S3 থেকে presigned URL generate করে (মেয়াদ যেমন ১ ঘণ্টা)
  9. Browser সরাসরি সেই presigned URL দিয়ে MinIO থেকে ফাইল download করে (backend server ফাইল proxy করে পাঠায় না – এটা bandwidth ও latency বাঁচায়)

**Presigned URL** ব্যাপারটা গুরুত্বপূর্ণ: এটা S3/MinIO এর একটা ফিচার যেখানে backend একটা সাময়িক, sign করা লিংক বানায় যেটা নির্দিষ্ট সময় পর আর কাজ করবে না। এতে ফাইল private bucket এ থাকা সত্ত্বেও নির্দিষ্ট approved user সাময়িকভাবে access পায়, backend কে ফাইল আবার download করে পাঠাতে হয় না।

### 8.৩ Visualization Workflow

**Time-series (তুলনামূলক হালকা query):**

Spatial map / interpolated surface (ভারী compute):

1. User area (bounding box) + variable + date বেছে নেয়

2. POST /api/visualize/spatial

3. যদি হালকা হয় (ছোট area/অল্প পয়েন্ট): সরাসরি backend এ synchronously process করে GeoJSON/raster ফেরত দেয়

4. যদি ভারী হয় (বড় area, interpolation দরকার):

a. Celery job queue হয়, API সাথে সাথে job\_id ফেরত দেয়

b. Worker: xarray/scipy দিয়ে interpolation (kriging/IDW) করে, output GeoTIFF/PNG বানায়, MinIO তে সেভ করে

c. Frontend job\_id দিয়ে poll করে বা status check করে, শেষ হলে result URL পায়

5. Frontend Mapbox GL/Leaflet এ raster layer বা GeoJSON overlay করে দেখায়

8.8 Admin Dataset Management

Add new dataset:

Admin form পূরণ করে (name, description, category, variables) → datasets টেবিলে row তৈরি

ফাইল upload করে → MinIO তে raw/ এ যায় → dataset\_files এ row তৈরি

→ Celery background job: processed/ এ optimized ফাইল বানায়, metadata (columns, spatial/temporal extent) auto-detect করে dataset\_files/datasets টেবিল আপডেট করে

Update dataset:

Metadata edit করলে শুধু datasets টেবিল আপডেট (ফাইল অপরিবর্তিত)

নতুন version এর ফাইল হলে dataset\_files এ নতুন row (version bump), পুরনোটা রেখে দেওয়া যায় history এর জন্য

Add file to existing dataset:

dataset\_id ঠিক রেখে নতুন ফাইল upload → নতুন dataset\_files row

Delete dataset:

soft delete recommended (status='archived') আসল delete এর বদলে, যাতে ভুলে মুছে গেলে ফেরত আনা যায়

সত্যিকারের delete চাইলে: DB row মুছার আগে MinIO থেকেও object delete করতে হবে (নাহলে storage তে orphan file থেকে যাবে)

৫. Security

| স্তর           | ব্যবস্থা                                            |
|----------------|-----------------------------------------------------|
| Authentication | JWT token, hashed password (bcrypt)                 |
| Authorization  | Role-based (user/admin), প্রতি request এ role check |

| স্তর             | ব্যবস্থা                                                                                     |
|------------------|----------------------------------------------------------------------------------------------|
| Download access  | শুধু approved request এর জন্য presigned URL, মেয়াদ সীমিত                                    |
| Storage          | MinIO bucket private, শুধু backend service credential দিয়ে access                           |
| Network          | Nginx এ HTTPS বাধ্যতামূলক (Let's Encrypt free SSL), 9000/9001 পোর্ট backend ছাড়া বাইরে বন্ধ |
| Rate limiting    | Nginx/FastAPI middleware দিয়ে API abuse ঠেকানো                                              |
| Audit log        | download_logs ও query_logs টেবিলে সব activity ট্র্যাক                                        |
| Input validation | Pydantic schema দিয়ে সব input strict validate                                               |
|                  |                                                                                              |

৬. Backup Strategy

- MinIO data** (`~/minio/data`): প্রতিদিন rsync/cron দিয়ে অন্য ডিস্ক বা off-site এ কপি (Hostinger backup addon, বা কম খরচে অন্য একটা ছোট VPS/cloud bucket এ sync)
- PostgreSQL**: `pg_dump` দিয়ে daily automated backup, cron job দিয়ে schedule, off-site এ পাঠানো
- সাধারণ নিয়ম**: কমপক্ষে ৩টা কপি, ২টা ভিন্ন storage medium এ, ১টা off-site

৭. Scaling Roadmap

| পর্যায়               | Storage                                 | Compute                                      | পরিবর্তন লাগবে কী                                                           |
|-----------------------|-----------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------|
| এখন<br>(১০-৫০GB)      | MinIO (VPS ডিস্ক)                       | একটা VPS, সব container একসাথে                | কিছুই না — এটাই baseline                                                    |
| মাঝারি<br>(কয়েকশ GB) | MinIO (বড় ডিস্ক attach) বা AWS S3 শুরু | Celery worker সংখ্যা বাড়ানো                 | শুধু <code>storage_backend</code> কনফিগ, কোড অপরিবর্তিত                     |
| বড়<br>(TB-১০০TB)     | AWS S3/GCS (lifecycle policy সহ)        | DuckDB থেকে Spark/Athena এ upgrade প্রয়োজনে | Query layer আপগ্রেড, storage config বদলানো, backend business logic একই থাকে |

মূল স্থাপত্যনীতি অপরিবর্তিত থাকে সব পর্যায়ে: **database শুধু metadata, storage backend independent, presigned URL দিয়ে secure delivery** — এই তিনটা principle ধরে রাখলে পুরো সিস্টেম redesign ছাড়াই বাড়তে পারবে।

৮. Deployment Summary (VPS তে)

docker-compose.yml এ একসাথে থাকবে:

- nginx (reverse proxy + SSL)
- frontend (Next.js build)
- backend (FastAPI)
- postgres (+ postgis extension)

- redis
- celery-worker
- minio

সবগুলো একই `(docker-compose up -d)` দিয়ে চালু হবে, একে অপরের সাথে internal Docker network দিয়ে কথা বলবে (শুধু nginx বাইরের world এর জন্য expose থাকবে)।

এই ডকুমেন্টটা আপনার পুরো প্ল্যাটফর্মের blueprint হিসেবে ব্যবহার করতে পারেন — development শুরু করার সময় ধাপে ধাপে (প্রথমে DB schema + auth, তারপর catalog, তারপর request workflow, তারপর visualization) বানানো সবচেয়ে ব্যবহারিক হবে।